

Database Architecture Decision

MongoDB Atlas vs Aurora PostgreSQL vs RDS PostgreSQL vs DynamoDB

A structured comparison for the EduProva EKS platform (main-web, freelance-web, backend microservices), covering fit, features, security, latency, and cost — prepared to support project presentation / interview discussion.

Prepared for	EduProva — Production Kubernetes Architecture
Scope	Database layer only (EKS application/infra unchanged)
Current DB in use	MongoDB Atlas (Replica Set, TLS, IP Whitelist)

1. Current Architecture Context

The EduProva production stack runs on Amazon EKS with three application services — **main-web**, **freelance-web**, and **backend** — scaled with HPA and Karpenter, deployed via Argo Rollouts (blue/green). All backend services are already integrated with **MongoDB Atlas** as the external managed database, using a replica set, TLS in transit, IP whitelisting, automatic backups, and auto-scaling. Any database decision has to be evaluated against this existing, working integration.

Layer	Current Setup
Compute	Amazon EKS, 2 managed nodes (t3x.large), Karpenter autoscaling
Services	main-web, freelance-web, backend (REST / JSON APIs)
Database	MongoDB Atlas — Replica Set, Auto Scaling, Automatic Backups
DB Security (current)	TLS in transit, IP Whitelist
Deployment	GitHub Actions → ECR → Argo Rollouts (Blue/Green)

2. Database Comparison Matrix

Criteria	MongoDB Atlas	Aurora PostgreSQL	RDS PostgreSQL	DynamoDB
Data model	Document (JSON)	Relational (SQL)	Relational (SQL)	Key-value / wide-column
Schema	Flexible / evolving	Fixed, strict	Fixed, strict	Flexible, simple
Best fit here	Courses, listings, profiles	Payments, orders, subscriptions	Moderate relational workloads	Very high-scale, simple access patterns
Scaling model	Horizontal (sharding) + vertical	Storage auto-scales; up to 15 read replicas	Vertical + limited read replicas	Near-infinite, fully automatic
Ops overhead	Low (fully managed)	Medium	Medium	Very low
Fit for EduProva today	Best — already integrated	Strong only if redesigned around transactions	Good but narrower than Aurora	Narrow fit — not a general choice

Both MongoDB Atlas and Aurora PostgreSQL are strong, production-grade choices. The right one depends on the shape of the data — not on one being generically “better.”

3. Why MongoDB Atlas Fits EduProva Today

- The backend services already speak MongoDB natively — drivers, queries, indexes, and the data model are all built around documents.
- Application objects (courses, freelance listings, user profiles) map naturally to JSON documents rather than normalized tables.
- Data shapes change as the product evolves — flexible schema avoids constant migrations.
- Switching to a relational database means re-writing the driver layer, data model, indexes, transaction logic, and backup/restore strategy — high risk, high cost, for a system already working.

What Atlas already provides

Feature	Why it matters for EduProva
Replica Set	Automatic failover — primary loss doesn't take the app down
Auto Scaling	Compute/storage scale with traffic, independent of Kubernetes HPA
Atlas Search	Full-text search for course / freelance job listings
Change Streams	Real-time events, e.g. notify on a new job posting
Automatic Backups + PITR	Point-in-time recovery, reduces data-loss risk

4. When Aurora PostgreSQL Would Be the Stronger Choice

Aurora becomes the better fit if EduProva's core data is **strongly relational and transactional** — payments, orders, subscriptions, multi-table integrity — rather than content-style documents.

- Strong relational integrity: enrollments → courses → instructors, or users → subscriptions → payments.
- Multi-step transactions that must succeed or fail together (e.g. deduct balance + insert payment record in one atomic transaction).
- Storage auto-replicates across 3 Availability Zones (6 copies), with up to 15 read replicas for read scaling.
- IAM database authentication — short-lived (15-minute) tokens instead of long-lived DB passwords, a strong AWS-native security story.
- Aurora Serverless is attractive if traffic is unpredictable — capacity scales with demand and bills only while active.

Practical middle path: keep MongoDB Atlas for content/listings/profile data, and introduce Aurora PostgreSQL (or RDS) only for a strictly transactional module such as payments or subscriptions, if/when that module exists. This is a common production pattern (polyglot persistence) and avoids a full migration.

5. Security Comparison

MongoDB Atlas

Layer	How it works
Encryption at rest	AES-256 on cluster storage and snapshots by default; customer-managed keys supported
Encryption in transit	TLS required for all client connections
Network isolation	Currently: IP Access List (public endpoint, encrypted). Recommended upgrade: VPC Peering / AWS PrivateLink for a private connection, avoiding the public internet path
Authentication	SCRAM / X.509 certificates
Authorization	Built-in and custom RBAC — e.g. main-web = read-only, backend = read-write
Auditing	Query/access logging available for compliance

Aurora PostgreSQL

Layer	How it works
Encryption at rest	AWS KMS-backed encryption
Encryption in transit	SSL/TLS
Network isolation	Runs inside your VPC; Security Groups restrict which resources can connect (e.g. only EKS backend SG → Aurora SG on port 5432)
Authentication	Standard DB credentials, or IAM database authentication (15-minute temporary tokens — no long-lived password)
Authorization	Native PostgreSQL roles and grants

6. Latency

Latency is driven mainly by physical distance between the application and the database, not by which database engine is used. The same rules apply to both options:

- Deploy the database in the **same AWS region** as the EKS cluster — a cross-region setup adds roughly 100–200ms of unnecessary round-trip time.
- **MongoDB Atlas:** use VPC Peering or AWS PrivateLink instead of the public IP-whitelist path — this removes the public internet hop and typically brings latency down to single-digit milliseconds.
- **Aurora PostgreSQL:** same-region + Security Group restricted access keeps latency low; Aurora's shared storage design keeps replica lag minimal under normal load. For global users, Aurora Global Database replicates across regions in under one second.
- Proper indexing matters more than the database choice itself — a poorly indexed query will be slow on either engine.

Avoid quoting an exact number (e.g. "10ms") without having actually measured it in the deployment — latency depends on region, network path, query shape, indexes, and connection pooling.

7. Cost

	MongoDB Atlas	Aurora PostgreSQL
Entry tier	M10 — ~\$0.08/hr (~\$57–58/mo) per node	Pay for compute (instance) + storage + I/O separately
Realistic production	3-node replica set — roughly 3x the single-node rate before backup/transfer (~\$170–200+/mo at M10 scale)	Storage auto-replicates across 3 AZs; billed for writer + reader instances + storage + IOPS
Variable traffic option	Flex tier (usage-based, capped ~\$30/mo) for low-traffic pre-production	Aurora Serverless — capacity scales with demand, billed only while active
Operational savings	No self-hosting: no patching, replica-set config, failover scripting, or backup tooling to build	No self-hosting: no PostgreSQL install/patch/HA scripting

Neither database is categorically cheaper — it depends on workload shape. For EduProva's current moderate, document-oriented traffic, Atlas's single bundled price (compute + storage + HA + backup) is typically simpler to reason about than Aurora's itemized compute/storage/IOPS billing.

8. My Final Recommendation

Keep MongoDB Atlas as the primary database for EduProva today. Introduce Aurora / RDS PostgreSQL only for a specific, strictly transactional module (e.g. payments or subscriptions) if and when that need exists.

Rank	Database	When it's the right call for EduProva
1	MongoDB Atlas	Now — application already built around it; document model fits courses / listings / profiles
2	Aurora PostgreSQL	If rebuilding from scratch, or adding a heavily transactional module (payments, orders, subscriptions)
3	RDS PostgreSQL	Moderate relational workload where Aurora's extra scaling/HA isn't needed
4	DynamoDB	Only for a narrow, high-scale, simple key-value access pattern — not a general fit here

One line to explain this

"I selected MongoDB Atlas because EduProva's current data model is document-oriented and the application is already integrated with it. Atlas gives us a fully managed database with replica-set based high availability, automated backups, encryption, TLS, network access controls, and auto-scaling. It deploy it in the same AWS region as our EKS workloads to minimize latency. If EduProva's data model were strongly relational, with heavy transactional and multi-table integrity requirements — such as payments or subscriptions — Aurora PostgreSQL would be a strong alternative, and could even be added alongside MongoDB for just that module."

EduProva Production Kubernetes Architecture and database-selection.